

# INFORMATIQUE 2

## Algorithmique 2 / Python

### Chapitre 9 :

Les fichiers (via modules internes et bibliothèques externes)

**Par:**

Pr. Mohamed-Amine Chadi

Email: [m.chadi@uca.ac.ma](mailto:m.chadi@uca.ac.ma)

# Introduction

# Définitions

Un fichier est un ensemble d'information enregistrées sur un support (disquette, disque dur, etc.).

Au contraire des variables, le fichier est un moyen de persister (sauvegarder) les informations après même l'arrêt du programme

Ils existent plusieurs types de fichiers (binaire, texte, image, audio, etc.)

Les principales manipulations sur un fichier sont :

- **Lire** un fichier existant
- **Ecrire** dans un fichier existant ou **Créer** un nouveau fichier et Ecrire dedans
- **Fermer** le fichier
- **Sauvegarder** le fichier

# La fonction prédéfinie dans Python : **open**

# Ouvrir un fichier

## Et mode d'ouverture

```
## lire un fichier existant
```

```
nom = "test1.txt"
```

```
mode = 'r'
```

```
f1 = open(nom, mode)
```

```
# f1 est un objet de type classe io (input-output) spécifique du text
```

```
print(type(f1))
```

```
<class '_io.TextIOWrapper'>
```

```
f1 = open("test1.txt", "r")
```

```
#
```

```
"r" mode lecture (read)
```

```
f2 = open("test2.txt", "w")
```

```
#
```

```
"w" mode écriture (write)
```

```
f3 = open("test3.txt", "a")
```

```
#
```

```
"a" mode ajout (append)
```

```
## lire un fichier existant
```

```
nom = "test1.txt"  
mode = 'r'
```

```
f1 = open(nom, mode)
```

```
# f1 est un objet de type classe io (input-output) spécifique du text  
print(type(f1))
```

```
<class '_io.TextIOWrapper'>
```

```
# On peut accéder donc aux méthodes et attributs de f1
```

```
print(f1.name)
```

```
test1.txt
```

```
txt = f1.read()  
txt
```

```
'1 2 3\na b c'
```

```
print(txt)    #pour le voir correctement
```

```
1 2 3  
a b c
```

```
f1.closed    #vérifier si le fichier à été fermé ou non
```

```
False
```

```
f1.close()  #si non, fermer le
```

```
f1.closed
```

```
True
```

Il est important de fermer un fichier après avoir terminé de l'utiliser pour plusieurs raisons :

**1.Libération des ressources système** : Lorsqu'un fichier est ouvert, le système alloue des ressources pour gérer ce fichier. Si le fichier n'est pas fermé correctement, ces ressources peuvent rester allouées inutilement, ce qui peut entraîner une utilisation excessive de la mémoire et des problèmes de performances.

**2.Conservation des données** : La fermeture d'un fichier garantit que toutes les données écrites ou modifiées sont correctement écrites sur le disque. Si un fichier n'est pas fermé et qu'un processus est interrompu avant que les données ne soient correctement écrites, cela peut entraîner une perte de données ou des fichiers corrompus.

**3.Prévention des conflits d'accès** : Dans certains cas, un fichier ouvert peut être verrouillé pour empêcher d'autres processus d'y accéder. En fermant correctement le fichier, vous libérez ces verrous et permettez à d'autres processus d'accéder au fichier si nécessaire.

**4.Respect des bonnes pratiques de programmation** : Fermer correctement les fichiers fait partie des bonnes pratiques de programmation en Python et dans d'autres langages. Cela garantit un code propre, lisible et évite les problèmes potentiels liés à la gestion des ressources.

```
with open('test1.txt', 'r') as f1:
    contenu1 = f1.read()
    print(contenu1)
```

```
1 2 3
a b c
```

```
f1.closed
```

```
True
```

```
f1.read()  #ne peut pas lire un fichier fermé !!
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[63], line 1
----> 1 f1.read()

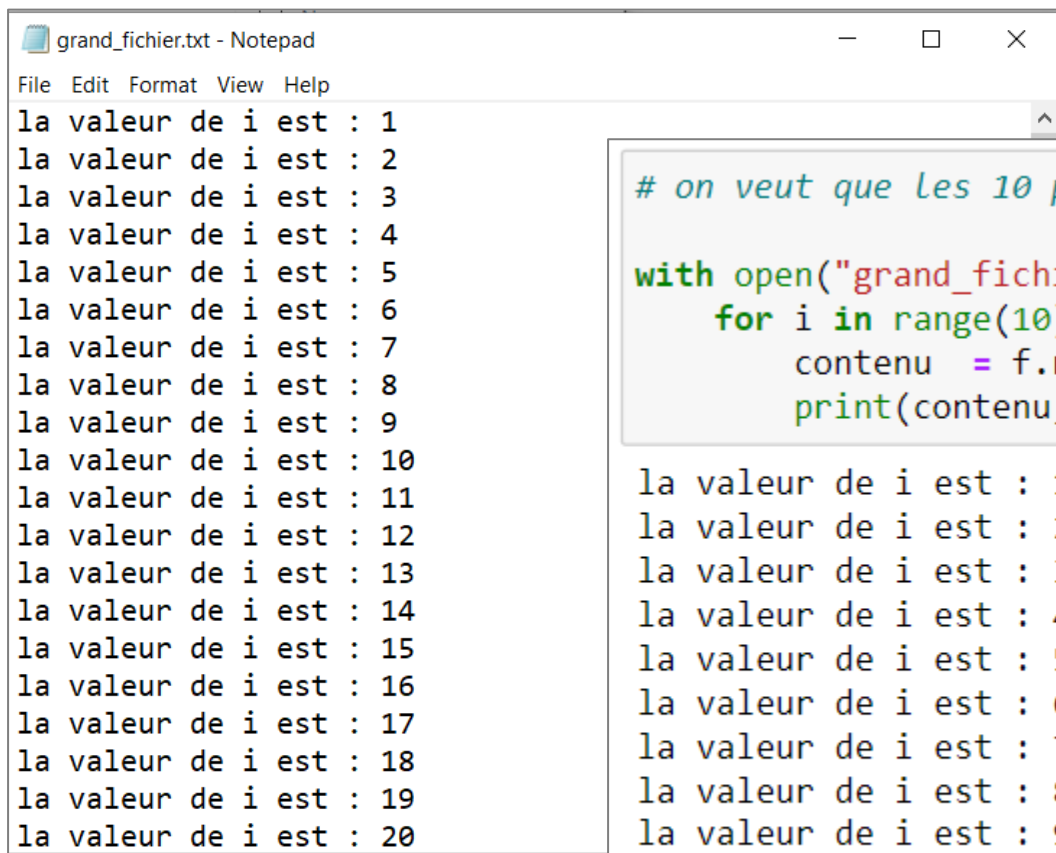
ValueError: I/O operation on closed file.
```

Lire le fichier en utilisant le mot-clé de Python **with**, aussi appelé « gestionnaire de contexte », dont parmi les avantages :

**Ferméture automatique du fichier :** Peu importe si le bloc de code à l'intérieur du **with** se termine normalement ou génère une erreur, Python appellera la méthode `__exit__()` du fichier. Cela garantit que le fichier est correctement fermé et que les ressources qui lui sont associées sont libérées, évitant ainsi les fuites de ressources et assurant une gestion efficace des ressources.

**si vous avez un fichier relativement grand en taille, vous pouvez ne pas lire tout en une seule fois**

- en utilisant `f.readline()`



```
grand_fichier.txt - Notepad
File Edit Format View Help
la valeur de i est : 1
la valeur de i est : 2
la valeur de i est : 3
la valeur de i est : 4
la valeur de i est : 5
la valeur de i est : 6
la valeur de i est : 7
la valeur de i est : 8
la valeur de i est : 9
la valeur de i est : 10
la valeur de i est : 11
la valeur de i est : 12
la valeur de i est : 13
la valeur de i est : 14
la valeur de i est : 15
la valeur de i est : 16
la valeur de i est : 17
la valeur de i est : 18
la valeur de i est : 19
la valeur de i est : 20
```

*# on veut que les 10 premiers éléments*

```
with open("grand_fichier.txt", 'r') as f:
    for i in range(10):
        contenu = f.readline()
        print(contenu, end='') #end='' parceque readline rajoute un '\n' automatiquement
```

```
la valeur de i est : 1
la valeur de i est : 2
la valeur de i est : 3
la valeur de i est : 4
la valeur de i est : 5
la valeur de i est : 6
la valeur de i est : 7
la valeur de i est : 8
la valeur de i est : 9
la valeur de i est : 10
```

## Ouvrir le fichier en mode écriture 'w' et en mode rajout 'a'

```
with open('test2.txt', 'w') as f4:  
    f4.write("abc")
```

Le mode 'w' remplace le contenu existant dans le fichier

```
with open('test2.txt', 'a') as f4:  
    f4.write("\n")  
    f4.write("faculté des sciences semlalia")
```

Le mode 'a' permet de rajouter du texte sans remplacer ou supprimer le contenu existant dans le fichier